



Freie Waldorfschule Leipzig

SPRACHASSISTENT MIT KI

Von Ferenc Dittrich

Mentoren: Stephan Dittrich, Stephan Wogatzke

Schuljahr: 2024/25

Abgabe: 01.10.2025



Inhaltsverzeichnis

1. Einleitung.....	2
2. Theorie	3
2.1 Was ist KI und woher kommt es?	3
2.2 Funktion und Technologie	6
2.3 Anwendung.....	8
2.4 Probleme.....	9
3. Praktischer Teil.....	10
3.1 Hardware	10
3.2 Software	12
3.3 Gehäuse	16
4. Fazit.....	18
5. Danksagung	18
6. Quellenverzeichnis	19
6.1 Praktischer Teil	19
6.2 Theoretischer Teil.....	20
7. Glossar.....	21
8. Eigenständigkeitserklärung	27

*„Ist es mir möglich einen Sprachassis-
tenten mit KI Funktionen zu bauen?“*

1. Einleitung

Vor etwa einem Jahr stellte ich mir die Frage, warum Sprachassistenten wie Siri, Alexa und Co immer noch keine Antworten auf alles haben, was man sie fragt. Oft klingen ihre Antworten weiterhin so, als wären sie lediglich vorprogrammiert. Dabei wäre es mit dem heutigen Technologiestand durchaus möglich, Künstliche Intelligenz in diese Systeme zu integrieren.

Ich beschäftige mich schon seit einigen Jahren mit dem Thema Künstliche Intelligenz. Außerdem programmiere ich viel und gerne in meiner Freizeit, wie zum Beispiel Computerspiele aber auch Automationen, welche einem in vielen Dingen die Arbeit erleichtern. So fragte ich mich, warum ich es nicht einfach selbst versuchen sollte? Und kam somit auf die Idee dieses Thema als Jahresarbeit zu nutzen. Damit habe ich ein Hobby von mir genommen und muss mir nicht alles von null auf neu beibringen. Ein Vorteil, den ich nutzen kann um etwas tiefer in das Thema einzusteigen und mehr Motivation zu finden.

Das Ziel war es einen **Smart Speaker** mit den Fähigkeiten eines **LLMs** zu bauen, mit der Möglichkeit ihn auch im Alltag zu nutzen.

2. Theorie

2.1 Was ist KI und woher kommt es?

Der Begriff „Künstliche Intelligenz“ bedeutet, dass ein **System**, welches nicht natürlich vorkommt, die Fähigkeiten wie Lernen, Probleme lösen oder logisches Denken, aber auch die Fähigkeit Sprache zu Erkennen oder visuelle Darstellungen wahrzunehmen, imitiert. ¹⁴

Im Jahre 1936 entwickelte der britische Mathematiker Alan Turing eine Theorie, des sogenannten Turing Tests. Sein Gedanke war, dass eine Rechenmaschine in der Lage sei, menschliches Verhalten nachzuahmen. Mit Hilfe des Turing Tests kann man herausfinden, ob eine vermeintliche künstliche Intelligenz als „Intelligenz“ bezeichnet werden kann. Dieser Test ist relativ simpel aufgebaut. ^{13 14}

Person A kommuniziert mit den Personen B und C, ohne physischen Kontakt zu ihnen zu haben. Jedoch ist entweder Person B oder C eine künstliche Intelligenz und Person A weiß nicht wer von den Beiden es ist. Wenn es nun Person A gelingt herauszufinden, wer Mensch und Wer Maschine ist, fällt die vermeintliche KI durch den Test. Gelingt es Person A jedoch nicht dies zu erkennen, hat die Maschine den Test bestanden und gilt als Intelligent. ^{7 13 14}

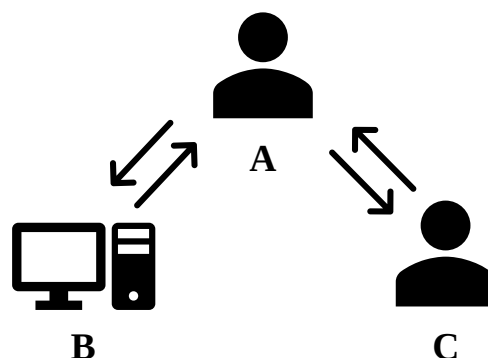


Abbildung 1: Turing Test

Zu diesem Zeitpunkt gab es den Begriff „künstliche Intelligenz“ jedoch noch gar nicht. Dieser wurde erst 1956 im Dartmouth College in New Hampshire erfunden. Während einer Konferenz wurde ein Programm entwickelt welches wir heute als

eine Art KI bezeichnen könnten. Einer der Programmierer, John McCarthy schlug daraufhin den Begriff „künstliche Intelligenz“ als Bezeichnung dafür vor, welcher sich bis heute hält.^{1 7 13 14}

Der erste **Chatbot** der Geschichte wurde bereits im Jahre 1966 entwickelt. Der deutsch-amerikanische Informatiker Joseph Weizenbaum programmierte ein Programm namens „ELIZA“, welches mit relativ einfachen Mitteln die Illusion eines menschlichen Gesprächspartners erzeugen konnte. Jedoch bestand es nicht den Turing Test. Es benutzte weder neuronale Netze, noch lässt es sich trainieren wie heutige Modelle. Ganz anders arbeitet es mit einer Art Wörterbuch. Es sucht sich wichtige Wörter (sogenannte Schlüsselwörter) aus der Eingabe des Benutzers heraus und ordnet sie zu verwandten Begriffen, wie zum Beispiel Eltern zu Familie, zu. In diesem Fall würde ELIZA etwas wie „Erzähl mir mehr über deine Familie.“ antworten. Der Code für ELIZA galt einige Zeit lang als verschollen, doch er wurde später in den Archiven des MIT wiedergefunden. Man kann ihn nun auf **GitHub.com** ansehen.^{3 7 13 14}

Erst ca. zwanzig Jahre später werden Modelle entwickelt, welche mit **neuronalen Netzen** arbeiten und es wird das erste Programm mit Sprachausgabe erfunden. Es trägt den Namen „NETtalk“ und wurde von Terrence J. Sejnowski und Charles Rosenberg programmiert. Dieses Programm ist dank neuronaler Netze in der Lage ihm auch unbekannte Wörter zu lesen und richtig auszusprechen.

Ab 2011 kommen künstliche Intelligenzen für den alltäglichen Gebrauch auf den Markt. Nach dem Sprachassistenten Siri von Apple, folgen weitere von Microsoft (Cortana, 2014) und Amazon (Alexa, 2015), welche bis heute genutzt werden. Sie suchen – ähnlich wie bei ELIZA – nach Schlüsselwörtern in der Eingabe (in diesem Fall aus der gesprochenen und in Text umgewandelten Sprache) und versuchen mithilfe von Informationen von Amazon oder Drittanbietern herauszufinden, was der Nutzer möchte, um daraufhin eine passende Antwort zu generieren.^{1 5 7 11 13}

Mit der Zeit werden leistungsstärkere Computer entwickelt und daher erscheinen auch immer bessere KI Modelle. Im Jahr 2012 gab es eine große Entwicklung im Bereich der Sprach- und Bilderkennung aber auch in der Automatisierung der Landwirtschaft. Diese Forschung zog sich eher unauffällig dahin bis zum sogenannten „KI Frühling“ in den frühen 2020ern. KI Systeme wie LLM`s oder **Diffusionsmodelle** werden kostenlos zur Verfügung gestellt, was einen KI-Boom in der Medienwelt auslöst. Vor allem ChatGPT ein LLM von OpenAI, welches im November 2022 veröffentlicht wird, ist nach kurzer Zeit überall bekannt und bereits nach 5 Tagen haben sich eine Million Nutzer registriert. Kurz darauf folgen weitere LLM`s von Microsoft (Bing Chat) und Google (Bard) und liefern sich einen Konkurrenzkampf.^{1 7 13 14}

Wie am Anfang des Kapitels schon erwähnt, „imitiert“ KI nur echte Intelligenz. Heutige Modelle, wie zum Beispiel ChatGPT, besitzen keine echte Intelligenz und haben somit auch kein echtes Bewusstsein. Sie können nicht eigenständig lernen oder sich neuen Umgebungen anpassen, also können sie nur eine vorgegebene Aufgabe erledigen und sind darauf spezialisiert. Diese nennt man ANI (Artificial Narrow Intelligence) oder umgangssprachlich auch schwache künstliche Intelligenz. Im Gegensatz dazu steht die sogenannte AGI (Artificial General Intelligence oder starke künstliche Intelligenz). Sie kann eigenständig lernen, kombinieren und flexibler denken, jedoch gibt es solche Modelle noch nicht. Zusätzlich gibt es noch eine dritte Art. Die ASI (Artificial Superintelligence oder künstliche Superintelligenz). Diese künstliche Intelligenz ist dem Menschen in allem überlegen. In strategischer- oder Kreativer Intelligenz aber auch sozialer Intuition. Aber sie existiert ebenfalls noch nicht und basiert auf reiner Spekulation.^{13 14}

2.2 Funktion und Technologie

Wie im vorherigen Kapitel schon festgestellt wurde, arbeiten KI Modelle mit sogenannten neuronalen Netzen. Die künstlichen neuronalen Netze von KI Modellen sind ähnlich aufgebaut wie die Funktion im Gehirn eines Menschen. Ein künstliches Netz ahmt **Neuronen** nach, welche durch viele Verbindungen zu einem Ergebnis kommen. Hier eine Visuelle Darstellung: ¹²

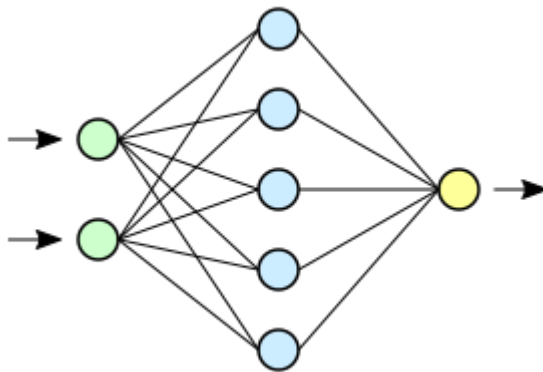


Abbildung 2: Visualisierung eines neuronalen Netzes

Jedoch ist diese Darstellung deutlich vereinfacht. Bei einem echten künstlichen neuronalen Netz gibt es deutlich mehr Ebenen und Knotenpunkte. Bleiben wir aber bei einem Vereinfachten Beispiel. ¹⁴

In einem neuronalen Netz gibt es drei Schichten. Die Eingabeschicht (Grün), die sogenannte verborgene Schicht (Blau) und die Ausgangschicht (Gelb). Die Eingabeschicht besitzt die Auswertungsdaten, die zum Beispiel vom Benutzer eingegeben wurden. Diese gehen über in die verborgene Schicht welche aus beliebig vielen Ebenen bestehen kann. Dort werden die Daten Ebene für Ebene verschieden oder stärker gewichtet, zum Beispiel Schlagwörter oder bei Bildern die Umrisse des Motivs. Jedoch ist der Prozess dieser Schicht nicht sichtbar, daher auch der Name. In der Ausgangschicht wird dann das finale Ergebnis ausgegeben. ^{6 8 14}

Wie die verborgene Schicht genau funktioniert, kann man an folgendem Beispiel gut erkennen. Als ursprüngliche Information haben wir ein beliebiges Bild und das neuronale Netz soll herausfinden, ob es eine Katze ist.

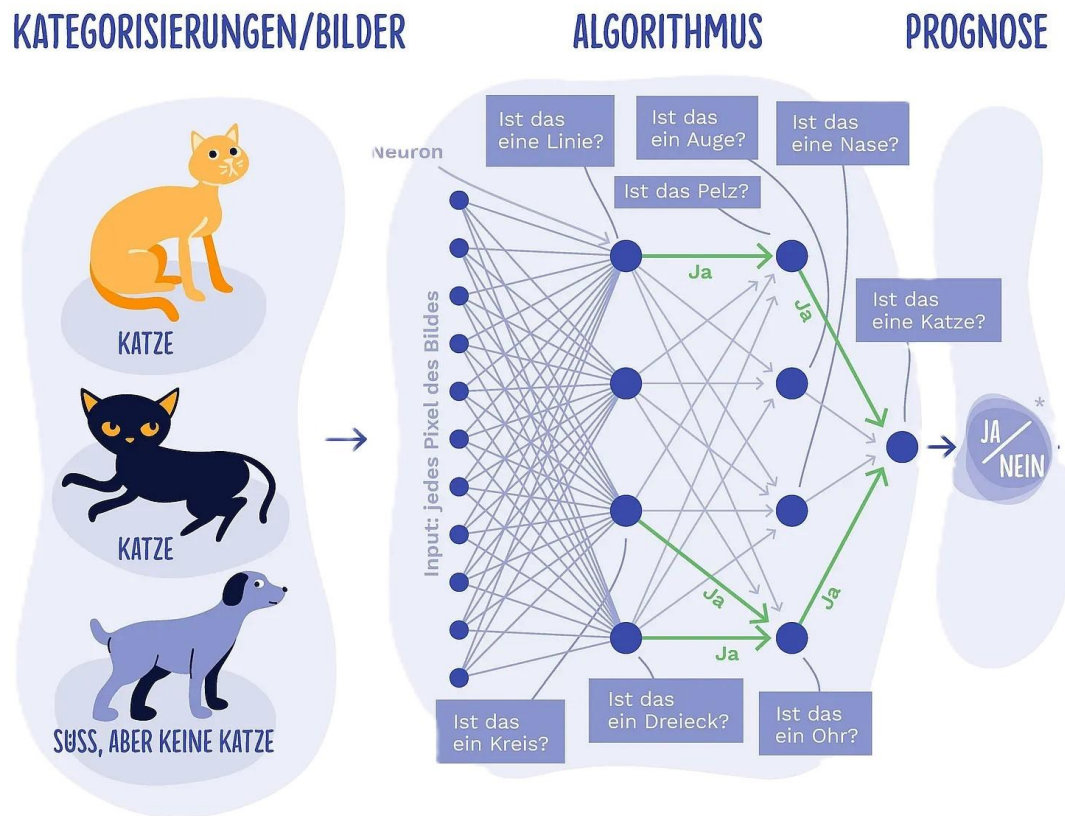


Abbildung 3: Funktion eines neuronalen Netzes²

Das gegebene Bild wird zunächst in dessen einzelne **Pixel** aufgeteilt. Diese Pixel werden nun auf besondere Merkmale durchsucht. Bilden sie zum Beispiel das Muster einer geometrischen Form, werden diese Neuronen gewichtet. Ebene für Ebene werden die Pixel detailreicher analysiert und somit eine Nase oder ein Ohr erkannt. In der letzten Ebene entscheidet eine sogenannte Aktivierungsfunktion, je nach gewichteten Neuronen, ob es eine Katze ist oder etwas anderes. ^{6 8 14}

Aber woher weiß das neuronale Netz denn eigentlich welche Merkmale es Beachten muss? Dafür müssen die Netze „trainiert“ werden. Zum Trainieren eines neuronalen Netzes gibt es drei Methoden. Das Supervised Learning (überwachtes lernen), das Unsupervised Learning (unüberwachtes Lernen) und das Reinforcement Learning (bestärkendes Lernen). In unserem Beispiel wurde Supervised Learning

benutzt. Bei dieser Methode werden (bei unserem Beispiel) Mengen an Bildern von Katzen oder anderen Motiven, inklusive der Bezeichnung des Motivs, dem Netz gezeigt (z.B. ein Bild einer Katze mit der Bezeichnung „Katze“). Damit kennt das Netz die Merkmale bestimmter Motive. Diese Art des Lernens wird z.B. bei der Spracherkennung, Bilderkennung aber auch bei Spamfiltern in E-Mail-Clients verwendet. Unsupervised Learning funktioniert ähnlich. Der einzige Unterschied besteht darin, dass die Daten nicht bezeichnet sind dadurch kann das Netz zwar ähnliche Motive oder Dateien filtern, aber diese nicht benennen. Beispiele dafür sind Datenanalysen oder die Erkennung von Anomalien. Reinforcement Learning ist wohl die spannendste Methode. Bei ihr lässt man dem Modell freien Lauf. Es probiert einmal etwas aus z.B. bewegt es sich in einem Videospiel nach rechts. Trifft es dann auf ein Hindernis, bekommt es eine „Bestrafung“ und weiß, dass es sich anders bewegen muss, um ans Ziel zu kommen. Diese Methode wird (wie schon im Beispiel) für Bots in Videospielen, in der Robotik oder für autonome Fahrzeuge verwendet. Selbst diese Erklärung ist sehr oberflächlich und zusammengefasst. Würde man dies ausführlich behandeln, könnte man allein damit ein ganzes Buch füllen. ^{6 8 14}

2.3 Anwendung

Wie vielleicht schon mitbekommen, ist nicht nur das, was man direkt als KI erkennen kann KI (wie z.B. ChatGPT). Künstliche Intelligenz wird in vielen weiteren Bereichen verwendet, welche einem vielleicht nicht direkt in den Sinn kommen.

Sie wird unter anderem in der Medizin als Diagnoseunterstützung, Medikamente Entwicklung und Patientenüberwachung, im Verkehr als autonome Fahrsysteme Verkehrsflussanalyse und Routenplanung, in der Industrie als Qualitätskontrolle, Robotersteuerung und Predictive Maintenance außerdem im Marketing bei Produktempfehlungen und personalisierter Werbung, in der Sicherheit für Gesichtserkennung, Verhaltensanalyse und Anomalie Erkennung und natürlich auch

in kreativen Bereichen wie Bild- und Musikererkennung, Texterstellung und Video-
produktion verwendet. ^{10 14}

2.4 Probleme

Auch wenn es manchmal so erscheinen mag, dass KI allwissend und nahezu perfekt ist, bringt sie so einige Probleme mit sich.

Da KI nur das weiß was in den Trainingsdaten steht, denkt sie auch, dass das alles richtig ist. Finden sich z.B. rassistische oder sexistische Äußerungen in den Trainingsdaten, so übernimmt die KI sie. Genauso funktioniert das mit Fehlinformationen oder ähnlichem. Dieses Phänomen nennt man „Bias“.

Umso besser KI wird, umso besser wird auch die Manipulation oder Desinformation welche man mit Hilfe von künstlicher Intelligenz erschaffen kann. Gemeint sind täuschend echte Texte, Bilder oder Videos, wie z.B. **Deepfakes**, um Falschinformationen zu verbreiten.

Um ein KI Model zu trainieren benötigt es sehr viele Daten von menschlichen Texten, Bildern, etc. Daher sind Unternehmen wie z.B. Meta dabei, Informationen von Instagram und Co dafür zu verwenden, was ein großes Datenschutzproblem hervorruft.

Insgesamt lässt sich festhalten, dass beim Einsatz von KI auch immer die Risiken bedacht werden müssen. ¹⁴

3. Praktischer Teil

Wie schon in der Einleitung erwähnt war es mein Ziel einen Smart Speaker mit den Fähigkeiten eines LLMs zu bauen, mit der Möglichkeit ihn auch im Alltag zu nutzen.

Vereinfacht gesagt einen Sprachassistenten wie Alexa, Siri und Co, welcher die Fähigkeiten einer KI ähnlich Chat GPT besitzt. Zusätzlich sollte er **Lokal** laufen, da man ihn somit offline nutzen kann und weniger Kosten für **APIs** anfallen.

3.1 Hardware

Zunächst habe ich mir überlegt, was ich konkret für Bauteile benötige und anschließend recherchiert, welche genau dafür in Frage kommen würden.

So sah dann das Ergebnis aus:

Funktion:	Eingabe	Verarbeitung	Ausgabe
Bauteile:	Seed-2Mic-Pi-Hat	Raspberry Pi 5	Mono Speaker

Als Gerät für die Verarbeitung habe ich den **Raspberry Pi 5** mit 8 GB Ram benutzt, das (zu der Zeit) neueste und leistungsstärkste Raspberry Pi Modell. Er hat ausreichend viel Anschlüsse für USB Geräte, Bildschirme, Kühlung, etc. (siehe Bild), benötigt aber ein spezielles USB-C Kabel für die Stromversorgung mit 5A/27W.

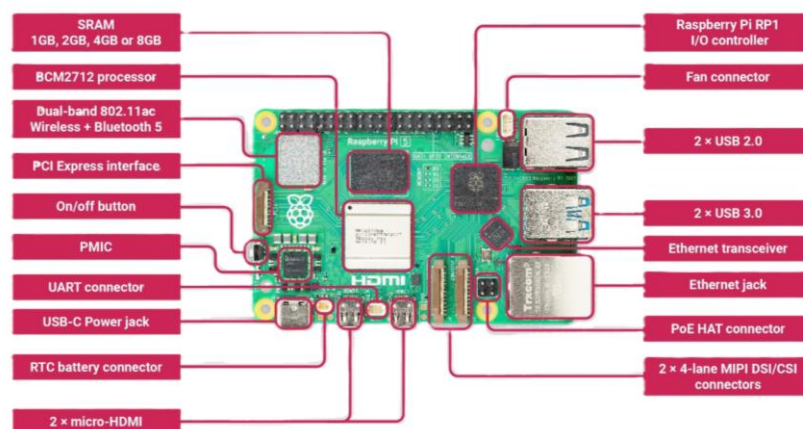


Abbildung 4: Raspberry Pi 5

Zusätzlich habe ich noch einen Active Cooler (Lüfter) montiert um die Leistungsfähigkeit zu verbessern. Als **Speichermedium** habe ich eine 1TB Toshiba Festplatte

mit USB Anschluss genommen.

Für die Eingabe habe ich mich für den **Seed-2Mic-Pi-Hat** entschieden, da er (dachte ich anfangs zumindest) sehr gut kompatibel mit dem Raspberry Pi ist. Jedoch galt dies ausschließlich für den Raspberry Pi 3 und 4. Der Seed-2Mic-Pi-Hat ist eine Platine mit 2 Mikros, einem Button, drei LED's und einigen weiteren Anschlüssen, welche man perfekt auf einen Raspberry Pi stecken kann. So auch bei meinem Raspberry Pi 5.

Das Problem stellte sich als **treibertechnisch** heraus heißt, der Treiber war noch nicht an das neue Raspberry Pi 5 Modell angepasst wor-

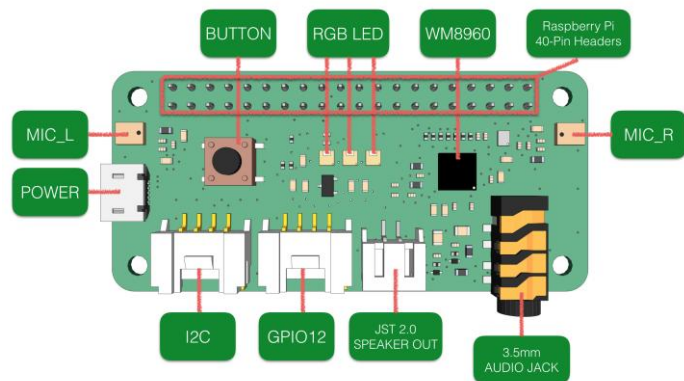


Abbildung 5: Seed-2Mic-Pi-Hat

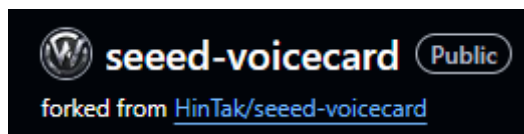


Abbildung 6: seeed-voicecard- Repository

son, welche exakt diesen Treiber mit dem Raspberry Pi 5 Modell kompatibel gemacht hatte. Dies hatte zwar einige Nachteile, wie dass z.B. die drei LED's nicht gingen aber ansonsten funktionierte es vorerst. Leider kostete mich das sehr viel Zeit.

Als Ausgabegerät habe ich einen einfachen **Monoaudio** Lautsprecher benutzt, welcher an den JST 2.0 SPEAKER OUT Anschluss des Seed-2Mic-Pi-Hat passt.

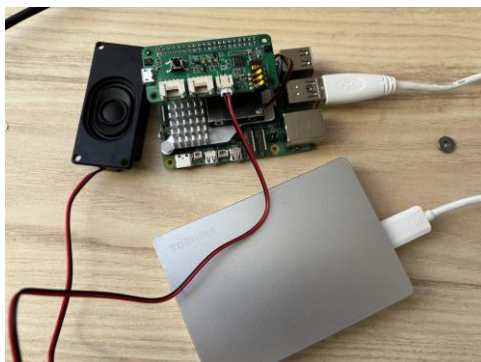


Abbildung 7: vorerst fertige Hardware

Wie erwähnt, bereitete die Soundkarte Treiberprobleme und selbst einige Zeit nach dem ich das inoffizielle Repository installiert hatte, gab es wieder Probleme



Abbildung 8: Mic+V3 Karte

und ich musste mir letztendlich eine Alternative suchen. Nach einiger Recherche stieß ich auf die MIC+V3 Karte von Raspaudio (Abbildung 8), eine ähnliche Soundkarte plus integrierten Lautsprechern. Sie wird offiziell vom Raspberry Pi 5 unterstützt und funktionierte relativ gut.

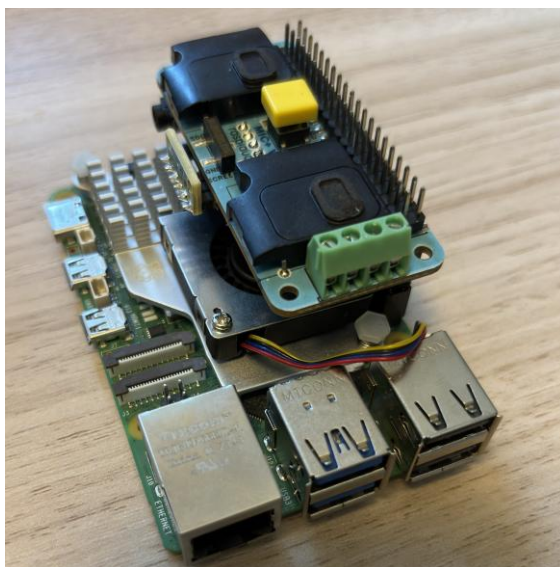
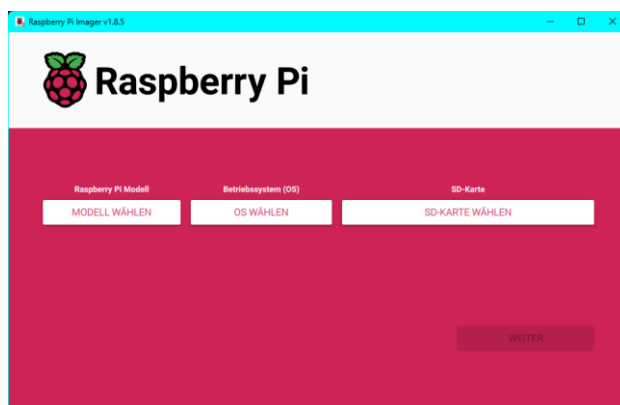


Abbildung 9: Mic+V3 Karte auf dem Raspberry Pi

5

3.2 Software

Um die Software schreiben zu können benötigte ich zunächst ein **Betriebssystem** auf meinem Raspberry Pi. Das machte ich über meinen Laptop mit dem Raspberry Pi Imager Programm. Das ist ein Programm mit welchem ich eine riesige Auswahl



an verschiedenen Betriebssystemen auf ein Speichermedium schreiben konnte um dieses dann mit dem Raspberry Pi zu verbinden und zu **booten** (Abbildung 10). Anfangs nutzte ich **Raspberry Pi OS**

Abbildung 10: Raspberry Pi Imager

(64-bit), ein **grafisches** Betriebssystem. Jedoch funktionierte die Soundkarte damit nicht und ich musste auf **Raspberry Pi OS Lite (64-bit)** umsteigen, das gleiche Betriebssystem aber ohne grafischer Oberfläche, also nur Text. Beim Beschreiben und booten des Betriebssystems nutzte ich noch ein anderes Kabel für die Stromversorgung, wodurch mein Raspberry Pi zu wenig Strom bekam und dadurch einige meiner **SD Karten** (welche ich zu dem Zeitpunkt als Speichermedium verwendete) unbenutzbar gemacht wurden. Nachdem ich es geschafft hatte, ein funktionierendes Betriebssystem zu booten, musste ich mir überlegen welche Programmiersprache ich nutze und in welchem **Editor**.

Die Entscheidung der Programmiersprache fiel mir relativ leicht. Ich nutzte **Python** eine einfache und sehr verbreitete Programmiersprache, welche gut kompatibel mit den meisten Tools und Ähnlichem ist. Hier als Beispiel für die **Syntax** ein einfaches **Hello-World-Programm**: `print ( "Hello, World" )` Dafür musste ich **Pip** installieren, damit ich diverse **Bibliotheken** installieren konnte. Zusätzlich installierte ich eine virtuelle Python Umgebung damit die Bibliotheken in keinen Konflikt mit dem Betriebssystem kommen.

Als Editor habe ich einen recht simplen Texteditor namens Nano genutzt. Nano ist ein vorinstallierter Texteditor für **Linux-Distributionen**.

Nun brauchte ich drei KI Modelle. Eins um Sprache in Text umzuwandeln, eins um die Antwort zu generieren und das dritte um die Antwort von Text in Sprache umzuwandeln.

Als Sprache zu Text Modell habe ich mich für **Vosk** entschieden. Es ist ein **Open-Source**-Modell, welches komplett lokal nutzbar ist und viele Sprachen unterstützt.

Für die Verarbeitung der Eingabe und zur Generierung einer Antwort habe ich ein **Llama** Modell verwendet. Llama Modelle sind Open-Source-Sprachmodelle (LLMs), und wurden von Meta entwickelt. Man kann sie über das **Ollama** System

nutzen und laufen ebenfalls komplett Lokal. Hierbei musste ich darauf achten, dass ich ein Modell nehme, welches nicht zu viel Leistung benötigt. Nach einiger Recherche habe ich mich für das Llama 3.2:1b Modell entschieden, ein stark **quantisiertes** und relativ kleines LLM. Leider waren dadurch die Antworten auch nicht sehr qualitativ.

Um die Sprache aus dem Antworttext zu generieren, benötigte ich ebenfalls ein Modell welches nicht zu viel Leistung verbraucht. Nach einigem Ausprobieren beschloss ich, dass **Pico TTS** die beste Wahl sei. Leider waren die Qualität und Natürlichkeit der Stimme dadurch ebenfalls nicht sehr gut.

All diese Komponenten musste ich nun in einem einzigen Programmcode zusammenbringen, dass sie miteinander funktionieren. Um nun mit dem Assistenten zu interagieren, muss man lediglich dieses Programm ausführen, und er hört direkt zu. Sobald man eine Pause mit dem Reden macht, antwortet er und beginnt kurz darauf, erneut zuzuhören.

Auf der folgenden Seite ist der komplette Code mit Kommentaren zum näheren Verständnis (Grüne Schrift) zu finden.


```

import ollama                                # KI-Kommunikation (Ollama)
import sounddevice as sd                     # Audioaufnahme
import queue                                 # Warteschlange für Audio
import sys                                   # Systemfunktionen (z. B. Fehlerausgabe)
import json                                  # JSON-Verarbeitung
import os                                    # Betriebssystembefehle
from vosk import Model, KaldiRecognizer     # Vosk-Spracherkennung
import pyaudio                               # Audiounterstützung

# --- Audio-Einstellungen ---
q = queue.Queue()                            # Warteschlange für Audio
samplerate = 48000                           # Abtastrate (Hz)
model = Model("/home/ferenc/vosk/vosk-model-de-0.21") # Sprachmodell laden

# --- Text-zu-Sprache (Pico TTS) ---
def speak(text):
    print("\n🗣️ Antwort wird gesprochen...")
    os.system(f'pico2wave -l=de-DE -w=/tmp/pico.wav "{text}" && aplay /tmp/pico.wav')

# --- Audio-Callback ---
def callback(indata, frames, time, status):
    if status:
        print(status, file=sys.stderr) # Fehlerstatus ausgeben
    q.put(bytes(indata))                # Audio in Warteschlange legen

# --- Sprache → Text ---
def recognize_speech():
    rec = KaldiRecognizer(model, samplerate) # Erkennen initialisieren
    with sd.RawInputStream(device=None, samplerate=samplerate, blocksize=0,
                           dtype='int16', channels=1, callback=callback):
        print("👂 Sprich jetzt (Pause zum Beenden)...")
        while True:
            data = q.get()                # Nächster Audio-Block
            if rec.AcceptWaveform(data):   # Wenn vollständige Äußerung erkannt
                result = json.loads(rec.Result())
                return result.get("text", "") # Text zurückgeben

# --- KI-Antwort holen ---
def ask_llama(prompt):
    print("\n🦙 Llama3 antwortet:\n")
    full_answer = ""
    response = ollama.chat(
        model='llama3.2:1b',
        messages=[{'role': 'user', 'content': prompt}],
        stream=True
    )
    for chunk in response:                 # Antwort-Stream verarbeiten
        token = chunk['message']['content']
        print(token, end='', flush=True)   # Live-Ausgabe
        full_answer += token
    print("\n")
    return full_answer

# --- Hauptprogramm ---
if __name__ == '__main__':
    while True:
        user_input = recognize_speech()    # Spracheingabe → Text
        if not user_input:
            print("⚠️ Nichts erkannt. Bitte nochmal sprechen.")
            continue

        print(f"🗣️ Du hast gesagt: {user_input}")
        answer = ask_llama(user_input)      # Text → KI-Antwort
        speak(answer)                      # Antwort ausgeben

```

Abbildung 11: Programmcode

3.3 Gehäuse

Für das Gehäuse wollte ich etwas schlichtes aber dennoch ansprechendes. Ich beschloss es 3D zu drucken, jedoch besitze ich keinen 3D-Drucker und ein Auftrag im Internet kostet relativ viel. Somit bat ich meinen Bruder Robin, dass er es mir an seiner Arbeitsstelle drucken lässt.

Also erstellte ich ein 3D-Modell auf womp.com und schickte es als zwei obj.-Dateien (Box und Boden) an meinen Bruder.

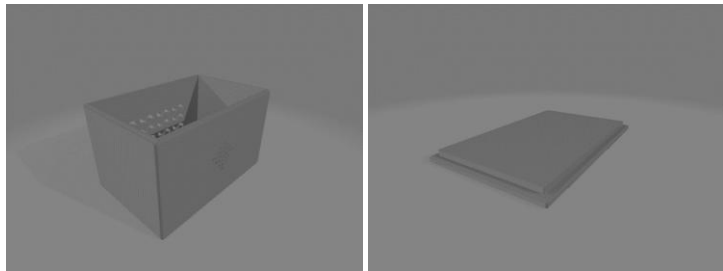


Abbildung 12: 3D Modelle des Gehäuses

Das Ergebnis sah folgendermaßen aus:



Abbildung 13: 3D Druck des Gehäuses

Da mir die Farbe nicht wirklich gefiel, musste ich es lackieren.

Dafür musste ich zunächst die komplette Box mit Sandpapier abschleifen und mit der Körnung immer feiner werden, sodass ich eine relativ ebene und angeraute Oberfläche habe. Nach dem entstauben und entfetten, sprühte ich nacheinander zwei



Abbildung 14: 3D Druck mit Primer Filler

Schichten Primer Filler darauf, damit sich die, durch den 3D-Druck entstandenen Rillen etwas füllten (Abbildung 14).

Anschließend schliﬀ ich es noch einmal fein ab und entstaubte und entfettete es wieder, so dass ich drei Schichten petrolfarbenden Sprühlack auftragen konnte.

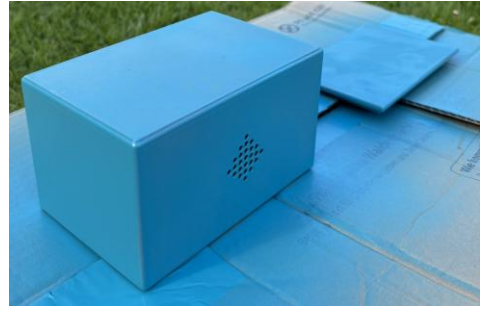


Abbildung 15: 3D Druck mit Sprühlack

Als kleines Detail habe ich auf Dortex.de zwei Labels aus Metall design und anfertigen las-



Abbildung 16: Metall Label

sen. Als Motiv habe ich mein Pseudonym, mein Logo und einen QR-Code zu meiner Webseite genommen (Abbildung links). Diese Labels habe ich

vorn und hinten an der Box angebracht.

Damit die Hardware nicht lose in der Box herum-
liegt, habe ich die Festplatte am Boden angeklebt
und den Raspberry Pi, mit Hilfe von alten Modell-
eisenbahn-Ersatzteilen, oben drauf befestigt (Ab-
bildung rechts).

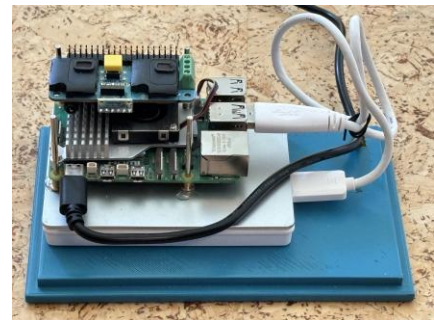
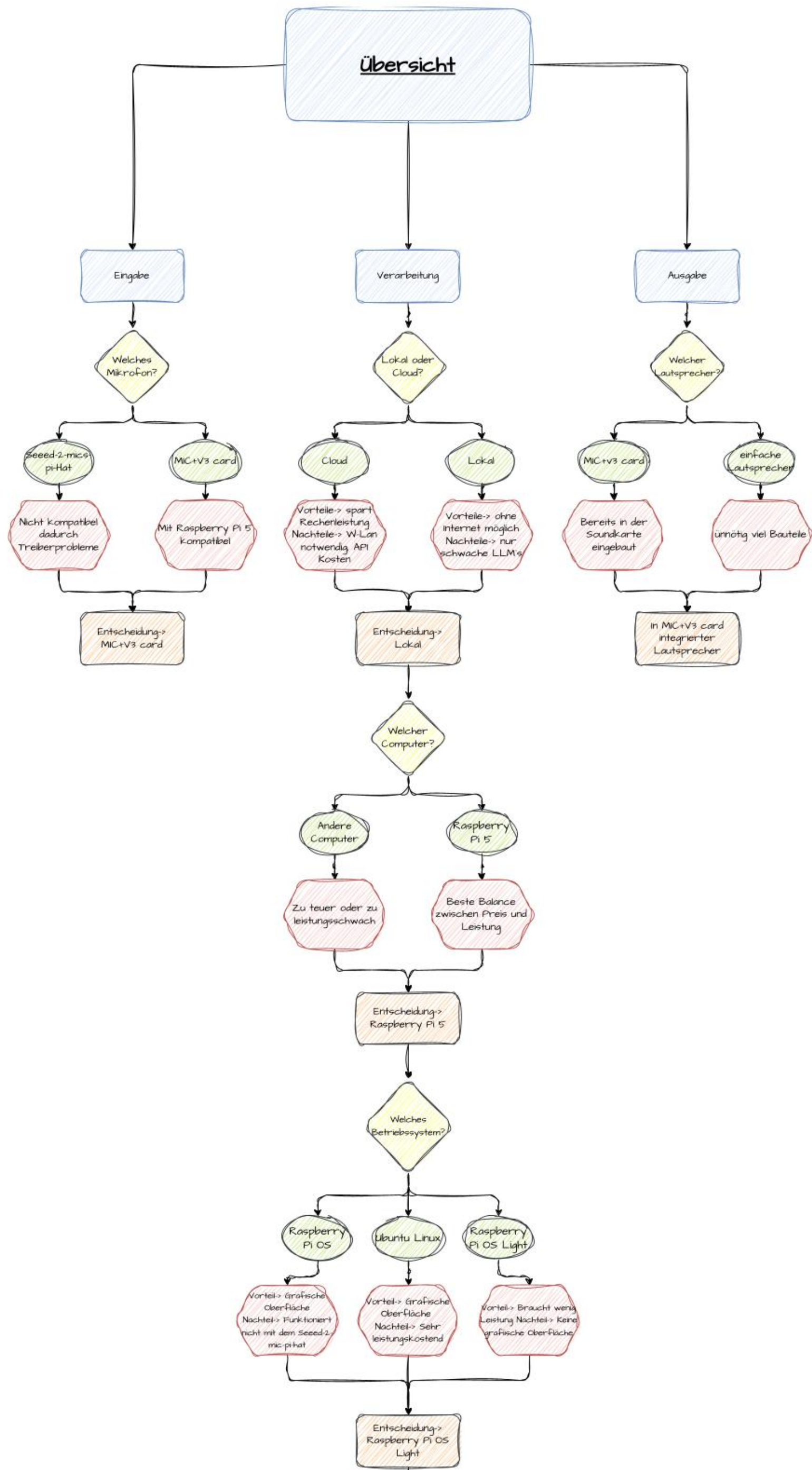


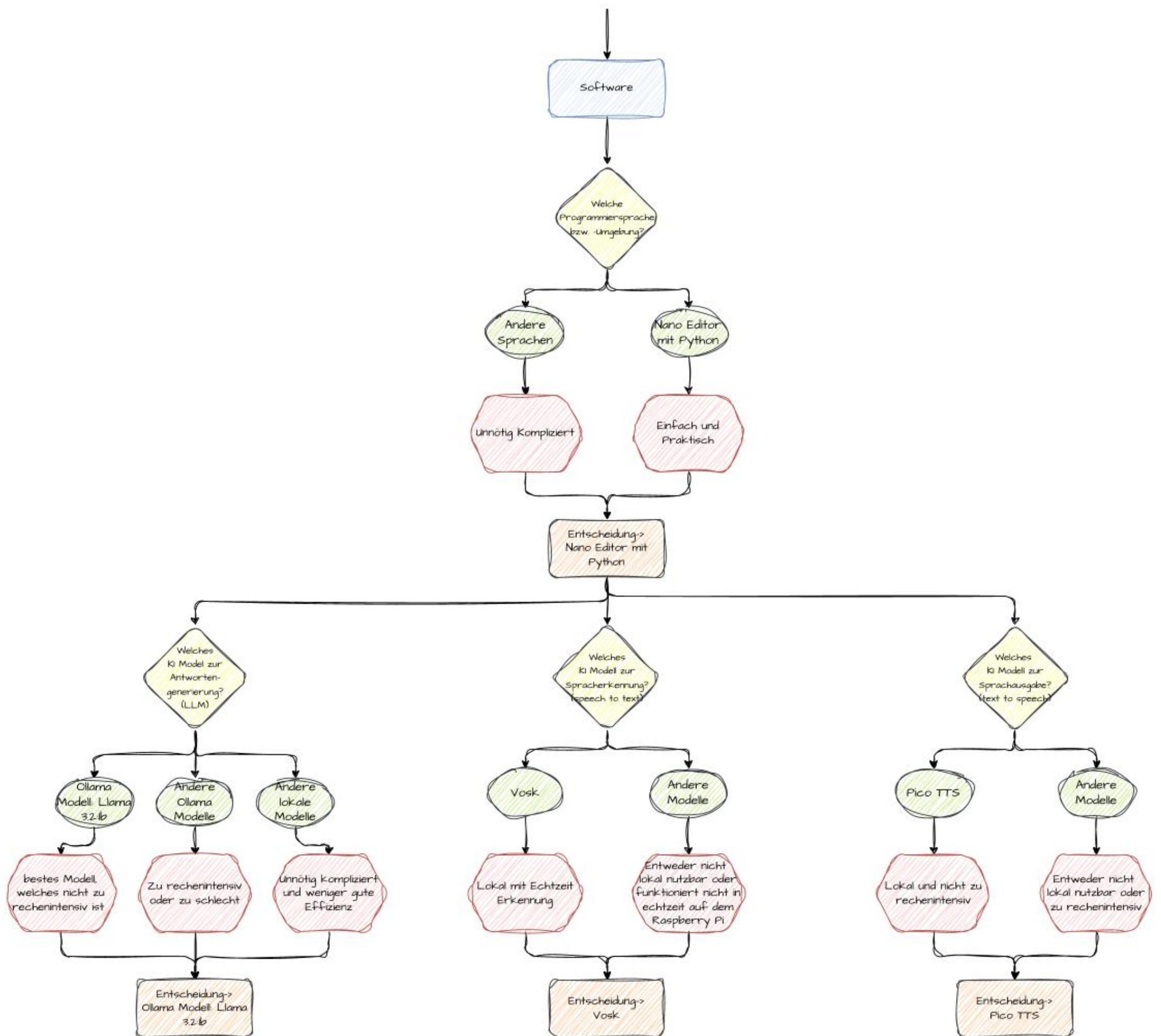
Abbildung 17: Befestigte Hardware



Abbildung 18: Finales Gehäuse

Auf den folgenden Seiten ist ein Flussdiagramm zum Entscheidungsprozess von Hard- und Software zu finden.





4. Fazit

Ich fand die Jahresarbeit ein sehr interessantes und aufschlussreiches Projekt. Auch wenn es relativ schwierig, und das Ziel vielleicht etwas zu hoch gesetzt war. Außerdem war mir meine Vorerfahrung im Programmieren eine große Hilfe und ohne diese wäre dieses Projekt fast unmöglich gewesen. Alles in allem kann ich jedoch sagen, dass ich meine Fragestellung „Ist es mir möglich einen Sprachassistenten mit KI Funktionen zu bauen?“ mit ja beantworten kann, auch wenn das Ergebnis nicht ganz meinen anfänglichen Vorstellungen entspricht. Ich habe einen Sprachassistenten entwickelt, welcher mit Hilfe von einem LLM, Antworten liefert, welche jedoch qualitativ nicht optimal sind. Vielleicht werde ich dieses Ziel aber in Zukunft erreichen, wenn ich den Sprachassistenten weiterentwickle und andere Ansätze ausprobiere. Die These, dass es keine Sprachassistenten mit KI Funktionen auf dem Markt gibt, hat sich das Jahr über auch verändert, da die Technik sich sehr schnell weiterentwickelt und genau dieses Thema immer weiter in den Vordergrund rückt. Man hört immer mehr, dass Unternehmen wie OpenAI vorhaben ähnliche Geräte zu entwickeln. Zum Beispiel gibt es Vermutungen, dass Sam Altman (CEO von OpenAI) zusammen mit Jony Ive (ehemaliger Chef-Designer bei Apple) ein Gerät entwickeln, welches ein KI Assistent als kleines, alltägliches Objekt ist.⁹

5. Danksagung

Zunächst danke ich Stephan Wogatzke als Mentor und meinem Vater Stephan Dittrich für generelle Unterstützung. Außerdem danke ich meinem Bruder Robin Dittrich für den 3D-Druck, Magnus Madaras für die Festplatte und meiner weiteren Familie und Freunden für Tipps und Unterstützung.

6. Quellenverzeichnis

6.1 Praktischer Teil

Aleksandar Haber PhD. YouTube. How to install and Run Llama 3.2 1B and 3B LLMs on Raspberry Pi and Linux Ubuntu. 30.09.2024. <https://www.youtube.com/watch?v=ql7aXdQ3-68&t=277s>

Aleksandar Haber PhD. YouTube. Install and Run Llama 3.1 LLM Locally in Python and Windows Using Ollama. 10.08.2024. <https://www.youtube.com/watch?v=sdwTN3d-KIQ&t=118s>

Data Slayer. YouTube. I Ran Advanced LLMs on the Raspberry Pi 5!. 08.01.2024. <https://www.youtube.com/watch?v=Y2ldwg8xsqE&t=649s>

Data Slayer. YouTube. I Ran ChatGPT on a Raspberry Pi Locally!. 29.08.2023. <https://www.youtube.com/watch?v=N0718RfpuWE&t=27s>

Digital Spaceport. YouTube. Die Integration meines Smart Home Assistant Ai Voice ist im Gange. 14.12.2024. <https://www.youtube.com/watch?v=-XUWm7AV3OQ&t=1s>

FutureProofHomes. YouTube. Build a LOCAL ChatGPT Voice Assistant For Your Smart Home. 30.12.2023. <https://www.youtube.com/watch?v=pAKqKTkx5X4&t=306s>

Ian Wootten. YouTube. Using Ollama to Run Local LLMs on the Raspberry Pi 5. 17.01.2024. <https://www.youtube.com/watch?v=ewXANEIC8pY&t=163s>

Jakerella. Trouble getting voice bonnet to work with Raspberry Pi. 30.03.2024. <https://forums.adafruit.com/viewtopic.php?t=208586>

LOGIN TechBlog. YouTube. ChatGPT lokal auf Raspberry Pi 5 - KI mit Ollama & Openweb UI mit Meta Llama 3.2. 17.11.2024. <https://www.youtube.com/watch?v=3VaLMWR38II&t=561s>

Matt Williams. YouTube. Unlocking The Power Of AI: Creating Python Apps With Ollama!. 01.04.2024. <https://www.youtube.com/watch?v=4K20tOsXK8&t=367s>

NetworkChuck (2). YouTube. Home Assistant made their own Alexa!!. 19.12.2024. <https://www.youtube.com/watch?v=An4lapvutzM&t=10s>

NetworkChuck. Create a Local, AI Voice Assistant (Alexa Replacement). 04.11.2024. <https://academy.networkchuck.com/blog/local-ai-voice-assistant>

NetworkChuck. YouTube. Run your own AI (but private). 12.03.2024. https://www.youtube.com/watch?v=WxYC9-hBM_g&t=419s

NetworkChuck. YouTube. your home automation SUCKS!.. 25.10.2024. <https://www.youtube.com/watch?v=k02P5nghmfs&t=4s>

Prilchen. Raspberry Pi 5 Einstieg: Notwendiges Zubehör und erste Schritte. 03.11.2023. <https://prilchen.de/raspberry-pi-5-einstieg-was-braucht-ich-fuer-den-start/>

Raspiaudio. Raspiaudio. MIC+ installation guide. 27.04.2025. <https://forum.raspiaudio.com/t/mic-installation-guide/17>

Seed Studio. ReSpeaker 2-Mics Pi HAT. 11.01.2023. https://wiki.seeedstudio.com/ReSpeaker_2_Mics_Pi_HAT/

Seed Studio. YouTube. Get started with google assistant - ReSpeaker 2-Mics Pi HAT#New Product. 02.06.2017. <https://www.youtube.com/watch?v=Z3glbxnDCl>

Seed Studio. YouTube. ReSpeaker 2 Mics Pi HAT Alexa Demo. 23.07.2018. <https://www.youtube.com/watch?v=uTyDqjG5o8s>

Shen. Raspberry Pi black screens on reboot after updating. 29.04.2019. <https://raspberrypi.stackexchange.com/questions/96843/raspberry-pi-black-screens-on-reboot-after-updating>

techiesms. YouTube. Running ChatGPT on ESP32 with Audio Output 🎧 🎧. 04.06.2023. <https://www.youtube.com/watch?v=R-CZLimCcW8&t=171s>

Varun Gujjar. YouTube. Raspberry Pi4 Offline AI Voice Assistant using ReSpeaker, No Internet Needed. 19.09.2021. <https://www.youtube.com/watch?v=q6uGNm-DCNc>

Wartem. seeed-voiccard. 09.10. 2024. <https://github.com/Wartem/seeed-voiccard>

WGLabz. YouTube. Audio Input to Raspberry Pi using Respeaker HAT | First Look #11. 29.11.2017. <https://www.youtube.com/watch?v=aUos6bSJ1Vg>

6.2 Theoretischer Teil

¹ AK Bildung in der digital vernetzten Welt – Allgemeinbildung. Die Geschichte der KI. 02.06.2025. <https://mebis.bycs.de/beitrag/ki-geschichte-der-ki>

² Buch.informatik.cc. Künstliche Neuronale Netze. <https://buch.informatik.cc/ki/ms-bilder>

³ Dipl.-Ing. (FH) Stefan Luber. Was ist ELIZA?. 30.01.2024. <https://www.bigdata-insider.de/was-ist-eliza-a-0615a5b88923975bed5af5c317fd1232/>

⁴ Hugging Face. Models. <https://huggingface.co/models>

⁵ Mathias Diwo. Ist Alexa eine künstliche Intelligenz?. 31.07.2024. <https://technavigator.de/technologie/kuenstliche-intelligenz/ist-alexa-eine-kuenstliche-intelligenz/>

⁶ Microsoft-Iskender Dirik. Was ist denn bitte Deep Learning?. 22.11.2028. <https://news.microsoft.com/de-de/deep-learning-whitepaper/>

⁷ Robert Bosch GmbH-Dr. Michael Schmidtke. Die Geschichte der künstlichen Intelligenz. 30.01.2018. <https://www.bosch.com/de/stories/geschichte-der-kuenstlichen-intelligenz/>

⁸ Studyflix. Neuronale Netze. <https://studyflix.de/informatik/neuronale-netze-4297/video>

⁹ The Wall Street Journal. Berber Jin. What Sam Altman Told OpenAI About the Secret Device He's Making With Jony Ive. 21.05.2025. <https://www.wsj.com/tech/ai/what-sam-altman-told-openai-about-the-secret-device-hes-making-with-jony-ive-f1384005>

¹⁰ Thws. Anwendungsbereiche der KI. <https://ki.thws.de/thematik/anwendungsbereiche-der-ki/>

¹¹ Tobias Westphal & Marcel Peters. Wie funktioniert Alexa? Einfach erklärt. 10.02.2025. https://praxistipps.chip.de/wie-funktioniert-alexa-einfach-erklart_97318

¹² TuringBot. Neural networks: what are the alternatives?. <https://turingbotsoftware.com/blog/neural-network-alternatives/>

¹³ Wikipedia. Geschichte der künstlichen Intelligenz. 03.06.2025. https://de.wikipedia.org/wiki/Geschichte_der_k%C3%BCnstlichen_Intelligenz

¹⁴ Wikipedia. Künstliche Intelligenz. 17.08.2025. https://de.wikipedia.org/wiki/K%C3%BCnstliche_Intelligenz

7. Glossar

Begriff	Definition
API	Eine Schnittstelle zur Kommunikation und Funktionsnutzung zwischen Programmen und/oder Servern.
Betriebssystem	Eine Grundlegende Software, die die Hardware eines Computers steuert, Ressourcen verwaltet und die Ausführung von Programmen ermöglicht.
Bibliotheken	Bibliotheken beim Programmieren sind Sammlungen von vorgefertigten Modulen und Funktionen, die bestimmte Aufgaben erleichtern (z.B. Datenanalyse oder Mathematik)
Booten	Der Startvorgang eines Computers oder eingebetteten Systems, bei dem das Betriebssystem geladen und das Gerät betriebsbereit gemacht wird.
Chatbot	Ein Computerprogramm, das in natürlicher Sprache mit Menschen kommunizieren kann, meist über Text oder Sprache.
Deepfakes	Ein Künstlich erzeugtes Bild-, Audio- oder Videomaterial, das mithilfe von KI so realistisch manipuliert wird, dass es echte Personen täuschend echt nachahmt.

Diffusionsmodelle	Eine Klasse von generativen KI Modellen, die schrittweise Daten (z. B. Bilder) aus Rauschen rekonstruieren, indem sie einen umgekehrten Diffusionsprozess lernen.
Editor	Ein Programm zur Erstellung und Bearbeitung von Texten oder Code. In der Informatik meist genutzt, um Skripte, Konfigurationsdateien oder Quellcode zu schreiben.
GitHub	Eine webbasierte Plattform zur Versionsverwaltung und gemeinsamen Entwicklung von Softwareprojekten.
Grafisch	Bezieht sich auf die Darstellung von Informationen in visueller Form – z. B. mit Bildern, Symbolen, Fenstern oder Benutzeroberflächen – im Gegensatz zu rein textbasierten Darstellungen.
Hardware	Die physischen Teile eines Computers oder elektronischen Geräts, wie Prozessor, Speicher, Mikrofone, Lautsprecher oder Platinen.
Hello-World-Programm	Ein einfaches Beispielprogramm, das meist nur den Text „Hello, World“ ausgibt. Es dient als Testprogramm einer Programmiersprache.
Linux-Distributionen	Verschiedene Varianten des Linux-Betriebssystems, die auf dem gleichen

	Kern basieren, sich aber in Zusatzprogrammen, Paketverwaltung und Benutzeroberfläche unterscheiden.
Llama	Abkürzung für Large Language Model Meta AI. Eine Reihe großer Sprachmodelle, die von Meta (Facebook) entwickelt wurden
LLM (Large Language Model)	Ein großes, auf Textdaten trainiertes KI-Modell, das natürliche Sprache verstehen und erzeugen kann.
Lokal	Ausführung oder Speicherung erfolgt direkt auf dem eigenen Gerät, ohne Verbindung zu einem externen Server oder zur Cloud.
Monoaudio	Ein Audiosignal, das über einen einzigen Kanal übertragen wird, im Gegensatz zu Stereo mit zwei Kanälen.
Neuron	Ein Grundbaustein im Gehirn, der Informationen verarbeitet und weiterleitet.
Neuronale Netze	Rechenmodelle in der Künstlichen Intelligenz, die vom Aufbau des menschlichen Gehirns inspiriert sind.
Ollama	Eine Software, mit der sich Sprachmodelle (LLMs) lokal auf dem eigenen Computer oder Server ausführen lassen. Sie erleichtert das Laden,

	Verwalten und Ansprechen verschiedener KI-Modelle über einfache Befehle.
Pico TTS	Ein leichtgewichtiges Text-to-Speech-System für Linux, das geschriebenen Text in gesprochene Sprache umwandelt. Es ist ressourcenschonend und läuft komplett lokal.
Pip	Ein Paketverwaltungsprogramm für Python, mit dem man zusätzliche Bibliotheken und Module installieren, aktualisieren und verwalten kann.
Pixel	Das kleinste einzelne Bildelement eines digitalen Bildes oder Displays, das eine bestimmte Farbe und Helligkeit darstellt.
Python	Eine einfach zu lesende, weit verbreitete Programmiersprache, die sich besonders für Einsteiger eignet und oft in Bereichen wie KI, Automatisierung und Webentwicklung eingesetzt wird.
Quantisieren	Zahlen und Daten von KI Modellen werden mit geringerer Genauigkeit gespeichert um Berechnungen zu beschleunigen.
Raspberry Pi OS (64-bit)	Ein offizielles, speziell für den Raspberry Pi entwickeltes Betriebssystem, das auf Linux basiert und in der 64-Bit-

	Version mehr Arbeitsspeicher und Leistung nutzen kann.
Raspberry Pi OS Lite (64-bit)	Eine minimale Version des Raspberry Pi-Betriebssystems ohne grafische Benutzeroberfläche. Sie basiert auf Linux, ist besonders ressourcenschonend und wird über die Kommandozeile bedient.
Raspberry Pi 5	Ein kreditkartengroßer Einplatinencomputer.
Repository	Ein Speicherort für Dateien und deren Versionsverlauf, der vor allem in der Softwareentwicklung zur Verwaltung von Quellcode verwendet wird – lokal oder auf Plattformen wie GitHub.
SD Karten	Kompakte Speichermedien (Secure Digital), die häufig in Kameras, Smartphones oder Einplatinencomputern wie dem Raspberry Pi zur Datenspeicherung verwendet werden.
Seed-2Mic-Pi-Hat	Eine Erweiterungsplatine für den Raspberry Pi.
Smart Speaker	Ein Lautsprechersystem mit integrierter Sprachsteuerung, welches auf Sprachbefehle reagiert und verschiedene Funktionen wie Musikabspielen, Informationsabruf oder Smart-Home-Steuerung ausführt.

Speichermedium	Ein physisches oder digitales Gerät, auf dem Daten dauerhaft oder temporär gespeichert werden können.
Syntax	Die Grammatik einer Programmiersprache, die festlegt, wie Code geschrieben sein muss, damit er vom Computer verstanden und fehlerfrei ausgeführt werden kann.
System	Eine strukturierte Einheit aus Hardware, Software und Daten, die gemeinsam eine definierte Aufgabe erfüllen – etwa ein Betriebssystem, ein Serversystem oder Ähnliches.
Treiber	Eine Software, die es dem Betriebssystem ermöglicht, mit einer bestimmten Hardwarekomponente (z. B. Mikrofon, Drucker) zu kommunizieren und diese zu steuern.

8. Eigenständigkeitserklärung

Hiermit versichere ich, dass ich die vorliegende Arbeit selbstständig angefertigt, keine anderen als die angegebenen Hilfsmittel einschließlich des World Wide Webs und anderer elektronischer Medien benutzt und die Stellen der Arbeit, die im Wortlaut oder im wesentlichen Inhalt aus anderen Werken entnommen wurden, mit genauer Quellenangabe kenntlich gemacht habe.

01.10.2025